

A multiple comparative study of test-with development product changes and their effects on team speed and product quality

Steve Bannerman · Andrew Martin

Published online: 13 July 2010
© Springer Science+Business Media, LLC 2010
Editor: Andreas Zeller

Abstract Researchers have typically studied the effects of Test-First Development (TFD), compared to Test-Last Development (TLD), across groups or projects, and for relatively short durations. We defined Test-With Development (TWD) as more general than the fine-grained step of TFD, but also in contrast to the large-grained phase of TLD. With our definition, we performed a multiple comparative study to explore and describe TWD product changes, and the effects of those changes on two attributes related to team speed and two attributes related to product quality, within six long-term open-source projects. Our results indicate that when developers exercised some of their changes with automated tests, on average they made significantly larger changes over time while significantly reducing their product's complexity. And, when they exercised all of their changes with tests, on average they made significantly smaller changes over time. We interpret these results to indicate that practicing TWD supports faster simplification of a product. Therefore, we conclude that teams that need to reduce their product's complexity can benefit from practicing TWD.

Keywords Multiple comparative study · Test-with development · Team speed · Product quality

1 Introduction

Over the last decade, Kent Beck and his colleagues have pioneered the Extreme Programming (XP) Method, which is an Agile Method within the Agile Model family. An XP team *listens* to a customer story, writes a set of *automated tests* for

S. Bannerman (✉) · A. Martin
Computing Laboratory, University of Oxford, Wolfson Building, Parks Road, Oxford, UK
e-mail: steve.bannerman@comlab.ox.ac.uk

A. Martin
e-mail: andrew.martin@comlab.ox.ac.uk

that story, evolves the system's *design* and *code* until all of the tests are working, and then *integrates* those changes (Beck 1999a). By taking best practices to extreme levels, teams should be able to develop and deliver their product to their customers more quickly and at higher levels of quality (Beck 1999b).

Test-Driven Development (TDD) is one of XP's best practices, and possibly *the* key practice (Beck 1999a; Jeffries et al. 2001; Auer and Miller 2002). When a team practices TDD, they write automated tests *before* they write or change their code, to ensure that their changes cause the new tests to move from *failing* to *passing* and to add to the team's test assets. Because they write their automated product tests before they change their product, they practice the Test-First Development (TFD) step within TDD and XP.

In principle, teams executing TFD could develop their product more quickly and at a higher level of quality—it could focus them, removing excess from their activities and helping them leverage automated test assets to improve their internal and external product quality. This hypothesis has been the subject of some research, and the results have been mixed. One subset of the researchers has studied students to determine the effects of TFD. Typically, they have conducted controlled experiments and overall they have found that practicing TFD increased speed a little but had no effect on quality. Another subset of these researchers has studied professionals by conducting controlled experiments and case-studies. Usually, they have studied real-life projects, albeit over relatively short durations, and overall have found that practicing TFD was the cause of, or correlated with, slightly decreased speed and slightly or moderately increased quality. To date, the relevant research has assumed that a team either practices TFD or they do not, rather than considering that a team may practice it to a degree. We present an overview of the TFD research in Section 2.

1.1 Test-With Development

When practicing TFD, developers evolve their automated product tests before making their fine-grained product changes (Beck 2003), in contrast to practicing Test-Last Development (TLD). In TFD, developers add or change their automated product tests first. And, each of their product changes is typically fine-grained and corresponds to a particular feature. But, when practicing TLD, developers evolve most or all of their automated product tests after making a series of product changes, if they evolve their automated product tests at all. Usually, the series of product changes is large-grained and corresponds to many features. And, in another more traditional variant of TLD, testing specialists evolve manual or automated product tests and then execute them after the development team has developed the features. Thus, there are several potential differences between TFD and TLD: *when* the product changes are made, relative to the product test changes; *how many* features are developed; *who* develops the product tests, the developers themselves or testing specialists; and *what* type of product tests are developed, manual or automated.

Further, we and others have observed that teams and developers sometimes practice variants of TFD. Sometimes, they evolve their automated product tests and make their product changes at the same time, interleaving them, in support of a particular fine-grained feature. Alternatively, they might make their product changes first and then evolve their automated product tests, also in support of a particular fine-grained feature—some of those performing related studies have differentiated

this approach from TFD by referring to it as “iterative test-last” (George and Williams 2003; Pancur et al. 2003; Erdogmus et al. 2005; Janzen and Saiedian 2006; Sanchez et al. 2007). So, developers sometimes vary when they change the product and when they change the automated product tests, even if they are making fine-grained changes for a particular feature.

Hence, we define Test-With Development (TWD) as either of these alternative approaches, in addition to the TFD approach. That is, TWD is defined by the co-evolution of automated product tests and product changes, by the developers themselves, in support of a particular fine-grained feature and independent of whether the developer makes the product changes first or last or in an interleaved manner. We choose the “with” modifier because the changes to the product tests are integrated and delivered before or with the related changes to the product, to the rest of the team, usually via a configuration management system.

Finally, we also recognize that developers may practice TWD to a degree. That is, a developer might only develop automated tests for some of the elements of the product changes that they make, instead of developing automated tests for all of them. For example, a Java developer might only develop automated tests for half of the new and changed product methods that they develop when adding a new feature. To reinforce our definition of TWD, and to introduce some ideas that are pertinent to our research, we present a product development example in Section 3.

1.2 Multiple Comparative Study

Therefore, given the potential benefits of TWD, assumed from the potential benefits of the TDD practice and the TFD step, and some limitations of the TFD research, we were motivated to begin to answer this question: *What are the relationships between practicing TWD, to varying degrees, and some attributes related to team speed and some attributes related to product quality, when those practicing it are real-life software development teams, developing their product over the course of many years?*

If the effects on the attributes related to team speed and product quality are significantly different, depending on the degree to which a team has practiced TWD, then future teams may want to practice it (or not) and may want to practice it to a certain degree. Ultimately, knowledge of the differences could allow teams to work more quickly, or to deliver products of higher quality, or both. Given the resources that we allocate to buying and developing software, and the importance of high quality software products to society, this is an important question.

As a result of our motivation to begin answering this question, we conducted a multiple comparative study to explore and describe these relationships. We rebuilt and studied the product and automated product test revisions of six long-term open-source projects and then performed statistical analyses on the resulting data to compare the differences between the effects of practicing TWD. We elaborate on our motivation, our plan, and our protocol for this study in Section 4. Then, we present and interpret its results in Section 5. Following the results, we describe the primary threats to their validity, in Section 6.

In summary, our results indicate that, on average, the degree of TWD that developers practiced when making product changes resulted in some significant differences between the effects of those changes. In Section 7, we present our con-

clusions, compare them with others' prior research results, discuss their implications and limitations, and identify potential future work.

2 Test-First Development Research

Although several researchers have performed studies to investigate the effects of TFD on, or correlations of TFD with, specific attributes of team speed or product quality, the results of their studies have been mixed. Some have found that attributes of team speed increase or stay roughly the same when individuals or a team execute TFD. Others have found that attributes of team speed decrease, if only slightly. However, some have not found any effect on attributes of team speed. Further, some have found that attributes of either the internal or external product quality increase, to a varying degree. Finally, some have been unable to determine the effect of TFD on attributes of product quality.

We have summarized their studies and results in four tables. Table 1 summarizes the studies where students were their subjects. Table 2 summarizes the studies where professionals were their subjects. Table 3 summarizes the attributes of team speed and product quality that they analyzed in each study. Table 4 summarizes the development durations for each study's subjects. We refer the reader to the relevant publications for more specific details on these studies.

Overall, these researchers have found that there is a slight increase in speed when they study students that practice TFD. In contrast, they have also found that there is a slight decrease in speed and a slight to considerable increase in quality when they study professionals that practice TFD.

In addition to the mixed results, we noticed a few things about these studies that we wished to improve upon in our own. Most of these studies have been relatively short in duration: days and weeks. And, all of them have assumed that a team practices TFD or that they do not, rather than considering that a team may practice it to a degree. Finally, they have either contrasted groups in a controlled environment or they have compared similar but not identical real-life projects, rather than contrasting or comparing within a project.

While planning and executing our study, we sought to improve upon these limitations. We studied projects and their products, after they had been completed, which were developed over the course of months and years, rather than studying projects in a controlled environment, or in their context, over the course of a day or weeks. As well, we have observed that developers and teams practice TWD (which includes TFD), and that they practice it to varying degrees. And, we assume that

Table 1 Effects of test-first development—students

Publication	Type	Speed	Quality
Muller and Hagner (2002)	Controlled	No difference	No difference
Pancur et al. (2003)	Controlled	–	No difference
Erdogmus et al. (2005)	Controlled	Faster	No difference
Madeyski (2005)	Controlled	–	Lower
Janzen and Saiedian (2006)	Controlled	Faster	No difference
Gupta and Jalote (2007)	Controlled	Faster	No difference

Table 2 Effects of/correlations with test-first development—professionals

Publication	Type	Speed	Quality
George and Williams (2003)	Controlled	Slower	Higher
Maximilien and Williams (2003)	Case study	No difference	Higher
Williams et al. (2003)	Case study	–	Higher
Geras et al. (2004)	Controlled	No difference	–
Abrahamsson et al. (2005)	Case study	No difference	–
Bhat and Nagappan (2006)	Case study	Slower	Higher
Canfora et al. (2006)	Controlled	Slower	–
Damm and Lundberg (2006)	Case study	–	Higher
Sanchez et al. (2007)	Case study	–	Higher
Siniaalto and Abrahamsson (2007)	Case study	–	Higher
Nagappan et al. (2008)	Case study	Slower	Higher

within a project, the confounding effects of contextual variables (Basili et al. 1986, 1999) will be dampened, therefore allowing for more reasonable comparisons. That is, within a project, its product, its process, its technology, and to a certain extent its people will vary less than they would across two or more similar projects.

3 Product Development Example

In this section, we present a product development example to introduce our software change model, or our frame of reference, which we developed to support our empirical study. The model itself, which we do not include in this paper because of its length, is available in a technical report (Bannerman and Martin 2010).

Consider a contrived software product, written in Java. Further, consider a single product class within that product, Amount, which could be used to model the

Table 3 Studied attributes of team speed and product quality

Publication	Team speed	Product quality
Muller and Hagner (2002)	Total time	Reliability
George and Williams (2003)	Productivity (time)	Acceptance tests
Maximilien and Williams (2003)	LOC per month	Defect rate
Pancur et al. (2003)	–	Code coverage, tests passed
Williams et al. (2003)	–	Defect density
Geras et al. (2004)	Total effort	Unplanned failures
Abrahamsson et al. (2005)	LOC, effort, productivity	–
Erdogmus et al. (2005)	Productivity (# of tests)	Acceptance test conformance
Madeyski (2005)	–	Acceptance tests passed
Bhat and Nagappan (2006)	Time estimates	Defect density
Canfora et al. (2006)	Time to complete	–
Damm and Lundberg (2006)	–	Fault rate
Janzen and Saiedian (2006)	Total effort	Internal metrics
Gupta and Jalote (2007)	Effort, productivity	Acceptance tests passed
Sanchez et al. (2007)	–	Defect density, complexity
Siniaalto and Abrahamsson (2007)	–	Program design, code coverage
Nagappan et al. (2008)	Time estimates	Defect density

Table 4 Development durations for study subjects

Publication	Development duration
Muller and Hagner (2002)	2 months
George and Williams (2003)	1 day (or less)
Maximilien and Williams (2003)	7 months
Pancur et al. (2003)	5 months
Williams et al. (2003)	7 months
Geras et al. (2004)	1 day (or less)
Abrahamsson et al. (2005)	9 weeks
Erdogmus et al. (2005)	26 hours (or less)
Madeyski (2005)	8 weeks
Bhat and Nagappan (2006)	7–12 months
Canfora et al. (2006)	1 day (or less)
Damm and Lundberg (2006)	12–18 months
Janzen and Saiedian (2006)	1 day (or less)
Gupta and Jalote (2007)	3 weeks
Sanchez et al. (2007)	5 years
Siniaalto and Abrahamsson (2007)	9 weeks
Nagappan et al. (2008)	7–12 months

attributes and behavior for an amount of money. Listing 1 shows a single product method for this class.

Listing 1 Product revision 1

```

public class Amount {
  public Amount add(Amount amount) {
    return new Amount(getValue() + amount.getValue());
  }
}

```

Within the product method, the developer has coded instructions that will be compiled for a (virtual) machine to execute. She has instructed the machine to get the receiver's value, get the other amount's value, to add them together, and then construct and return a new amount object with the result.

Now, assuming the team is maintaining an automated suite of product tests, a test class named AmountTest might look like Listing 2. It includes a simple test to ensure that the original Amount remains unchanged and that the new Amount has the correct value, within a “blue sky add” scenario.

Listing 2 Product tests revision 1

```

public class AmountTest extends TestCase {
  public void testBlueSkyAdd() {
    Amount srcAmt = new Amount(5.0);
    Amount dstAmt = srcAmt.add(new Amount(10.0));
    assertEquals(srcAmt.getValue(), 5.0, 0.0);
    assertEquals(dstAmt.getValue(), 15.0, 0.0);
  }
}

```

Because this test method will exercise the product method when it is compiled and executed, we consider the product method to be exercised by the product tests. Further, if the team is practicing TWD, they will have developed, integrated, and committed this test method *with* the product method, rather than do so after they have developed several other features.

Next, to make the product more useful, a particular developer modifies the Amount class so that amounts of other currencies can be added to an existing amount. He adds a new and more specific method, which accepts an exchange rate (Listing 3).

Listing 3 Product revision 2

```

public class Amount {
  public Amount add(Amount amount) {
    return new Amount(getValue() + amount.getValue());
  }
  public Amount add(Amount amount, double exchangeRate) {
    double exchangeValue = amount.getValue() * exchangeRate;
    return new Amount(getValue() + exchangeValue);
  }
}

```

Assuming that this developer did not modify the automated tests before committing the product changes, the original product method is still exercised by one test. However, the new product method is not. Therefore, in this case, the developer only practiced TWD to a degree.

In making the changes from product revision 1 (Listing 1) to product revision 2 (Listing 3), the developer added a method. At a more precise level, the developer added a certain number of instructions, both in the source code and in the resulting compiled instructions. And, since he did not delete any compiled instructions, the initial net size of his change is equal to the size of the compiled instructions that he added.

Finally, if we look at this class six months later, we might observe that the team has added a currency attribute to the Amount class. Thus, they may have removed the “new” method that had accepted an exchange rate because it was no longer necessary. The newest product method, which has the same signature as the original method, now determines the exchange rate by considering the currency of both amounts, the receiver and the argument (Listing 4).

Listing 4 Product revision N

```

public class Amount {
  public Amount add(Amount amount) {
    double exchangeRate = getExchangeRate(amount);
    double exchangeValue = amount.getValue() * exchangeRate;
    return new Amount(getValue() + exchangeValue);
  }
}

```

Now, if we reconsider the Amount class at revision 1 (Listing 1), we notice that the signature of the “add” method has remained unchanged, with respect to revision N (Listing 4). However, the compiled instructions within the method have changed. Some of the original compiled instructions have persisted, others have been discarded over time, and new ones have been added.

Similarly, if we reconsider the Amount class at revision 2 (Listing 3), we notice that the team has deleted the second method, with respect to revision N (Listing 4). Therefore, all of its compiled instructions were discarded over time. Thus, we can adjust the initial net size of the change by the discards over time to represent the net size over time for the change—in this case the net size over time is zero instructions per six months because all of the initial compiled instructions were deleted.

Further, we can compare the compiled class from revision 1 with the compiled class from revision 2 to determine the change in two of its attributes related to product quality: its number of potential bugs and its average method complexity. Findbugs is a static analysis tool that can be executed against a compiled class to determine the number of potential bugs within it—this tool does not find the number of defects that will actually be encountered by somebody executing the product; rather, it finds a number of potential defects within the product. Similarly, cyvis is a static analysis tool that can be executed against a compiled class to determine the cyclomatic complexity for each of its methods.

4 Study Design

In the following subsections, we use the recommendations and guidance from several authors, to communicate our study design (Pinsonneault and Kraemer 1993; Kitchenham et al. 1995; Basili et al. 1999; Lethbridge et al. 2005; Runeson and Host 2009). Each subsection describes a particular perspective of our study design: (1) its Motivation; (2) its Plan; and (3) its Protocol.

4.1 Motivation

In addition to being motivated to study the potential benefits of TWD (Section 1), and being motivated to improve upon some of the limitations of previous TFD studies (Section 2), multiple precedents of mining information from repositories motivated us to design and perform a similar type of study. Card et al. (1987) evaluated software engineering technologies using empirical and statistical methods. By identifying and matching past projects, they were able to derive productivity and reliability values from historical project data. Then, they compared the matched projects to evaluate the effects of particular technologies. And, Cook et al. (1998) proposed that exploratory empirical studies could be both useful and cost-effective. They suggested that organizations can correlate process “factors” and their effects, by using data from existing process and product repositories, And, since 2004, a community at the International Conference on Software Engineering (ICSE) has been working to evolve the practice of “mining software repositories” (Hassan et al. 2005; Diehl et al. 2005, 2006; Gall et al. 2007; Lanza et al. 2008). A basis for their work is that “software repositories contain a wealth of valuable information for empirical studies in software engineering” (Hassan et al. 2005).

4.2 Study Plan

4.2.1 Objective

Given our motivation in general, and our motivation to study historical data in particular, our objective was both exploratory and descriptive (Robson 2002; Runeson and Host 2009)—we could not manipulate the projects (and therefore not explain by proving causal relationships) and we were not trying to improve TFD or TWD. We wished to explore the relationships between the degree of TWD for product changes and the resulting effects on some attributes related to team speed and product quality. We also wished to describe any such relationships, if they existed. Additionally, assuming that our study might identify significant relationships, our other objective was to provide a basis for further research.

4.2.2 Research Question

Our research question, as introduced in Section 1, is: *What are the relationships between practicing TWD, to varying degrees, and two attributes related to team speed (initial net size and discards over time) and two attributes related to product quality (potential bugs and average method complexity), when those practicing it are real-life software development teams, developing their product over the course of many years?*

Given the existing TFD research results, which have been mixed and are described in Section 2, and the lack of any previous TWD results, our initial hypotheses were based solely on our experience in the software industry. We expected that:

- *H1*: Increasing the degree of TWD would correspond with a positive effect on the attributes related to team speed, because the increased amount of automated tests would give the developers the confidence necessary to make bigger changes over time.
- *H2*: Increasing the degree of TWD would correspond with a positive effect on the attributes related to product quality, because the increased amount of automated tests would give the developers the confidence necessary to increase the internal quality of their code.

4.2.3 Frame of Reference

Although using theories to develop a research direction is not well established in software engineering (Hannay et al. 2007), we developed a theory for our study. The frame of reference, or theory, for our study is our software change model—in this precise set-theory model, we characterize changes to product and automated product test classes, methods, and compiled instructions. We introduced the model's central ideas in the Java example in Section 3, but do not present the model in its entirety, due to its length and detail. However, it is available in a related technical report (Bannerman and Martin 2010).

With our model, we had a basis to estimate the degree of TWD for a change as well as for measuring the two attributes related to team speed (initial net size and discards over time). We relied on previously defined theories and tools to measure the two attributes related to product quality (potential bugs and average method complexity).

4.2.4 Subject Definition and Selection Strategy

As illustrated in the Java example in Section 3, the subjects of our study are products as they change over time. And, although subjects should be selected intentionally (Runeson and Host 2009), we selected ours based on availability, as do many studies (Benbasat et al. 1987) and experiments (Sjoberg et al. 2005). Because open-source projects are publicly available and many have been in development for several or more years, we selected some of these products as our subjects.

At first, our selection criteria was simple: a Java project with more than two years of development history, that maintained both a product and a corresponding set of automated product tests. However, due to our tool set, and problems building projects using other build systems (primarily maven), our criteria became more refined: we also required that the project have its own ant (<http://ant.apache.org>) build file. We selected several cases from the Apache family of open-source projects (<http://www.apache.org>), and also selected a couple of projects that we were already familiar with, because of our work in industry.

4.2.5 Data Collection Methods

Because we were motivated to study multiple products and thousands of product revisions, we planned on utilizing partial or full automation. With automation support, we selected all of the self-contained commits—those commits that contained changes to product source code (and optionally changes to automated test source code) and which could be compiled.

Generally, we planned to perform a quantitative study by representing the changes to the attributes of the products with numbers (Fenton and Pfleeger 1997). Specifically, we performed a static analysis of each product revision and of the appropriate product revision pairs to estimate the degree of TWD that its developer practiced, as well as its initial net size, discards over time, change in the number of potential bugs, and change in average method complexity.

4.3 Study Protocol

Before we could measure product version changes, we needed to build a representation of each change that we were going to study. We also needed to be able to analyze the results.

For these needs, we utilized software tools. We were able to use several publicly available tools for some of the tasks. But, for some others, we needed to develop custom tools. In the following sections, we describe the tools and the protocol we used to build, measure, and analyze the product revisions of the products that we studied.

4.3.1 Data Collection Procedures

In a typical open-source project, multiple geographically distributed developers collaborate to build and evolve a software product (Raymond 1999). Usually, the team of developers shares and coordinates their work via a network and a configuration management tool, such as the Concurrent Versioning System (CVS) (<http://www.cvshome.org>) or the Subversion system (SVN) (<http://subversion.tigris.org>). Once a particular developer has added or changed a feature, and optionally

added or changed automated tests, he or she commits their changes so that the rest of the team and any continuous integration servers have access to them. The result of the commit is a new revision in the team's repository.

To build the revisions of a product, we used a specific version of the Sun JDK, a specific release version of the ant product, specific versions of any third party dependencies, and a combination of custom shell scripts and the ant build scripts provided by the teams themselves. First, we identified all of the revisions that included Java files in the product change set. Then, we used scripts to update our working copy from the team's repository and to clean and build a particular revision of the product and the product tests. We executed this sequence for each pertinent revision and stored the resulting artifacts, mapped by revision number.

Because the team's product and their build scripts evolved during the lifetime of their project, we had to keep our shell scripts in sync with them. When a build failed, we had to analyze why it had failed and then change our custom shell script to incorporate the correct versions of the JDK, ant, and the thirdparty dependencies, and to select the team's ant tasks which were intended to compile their product and their automated product tests. Thus, unfortunately, we could only partially automate this process.

Once we had rebuilt the product revisions and automated product test revisions for a product, we were able to use our custom and thirdparty tools to measure and estimate values for the degree of TWD, the initial net size, the discards over time, and the changes to the number of potential bugs and the average method complexity. We describe our procedure for doing so in the following subsections.

Independent Variable—Degree of Test-With Development As illustrated in our Java example (Section 3), and defined in our software change model, we assume that the degree to which a developer practices TWD, for a particular change set, will be reflected by the relationships between the automated product tests and the product methods within that change set.

To measure the degree of TWD for a change, we developed and used a custom tool called jeanda (Java Efficiency and Agility) (<http://jeanda.tigris.org>). It leverages the Byte-Code Engineering Library (BCEL) (<http://jakarta.apache.org/bcel>) to model and compare revisions of compiled classes, methods, and instructions of Java code. First, it determines the compiled methods that have changed within a product revision pair. Second, it determines which of those methods were referenced by any of the compiled instructions within the automated product test methods, directly or indirectly. Then, it determines the fraction of the changed product methods that were referenced, and presumed exercised, by tests.

If all of the changed product methods had automated test methods related to them, then we categorized the change set as "all." Likewise, if none of the changed product methods had automated test methods related to them, then we categorized the change set as "none." However, if some of the changed product methods had automated test methods related to them, then we categorized the change set as "some." These categories provided the basis for our static analysis in the present study; our future work section describes an alternative and more accurate approach for a dynamic analysis.

Zaidman et al. (2008) used an alternative static approach to determine whether or not a team was practicing TDD, in addition to analyzing the co-evolution of a

product and its product tests in more general terms. They inferred a relationship between product classes and product test classes based on a naming convention for the corresponding source files. They also inferred that changes to both of the files implied that the developer practiced TDD. However, we wanted a more precise and granular approach and thus chose to probe the relationships between the automated product tests and the product, within the compiled code.

Dependent Variable—Effect on Speed (Initial Net Size) With the Lines of Code (LOC) measurement, bigger changes per time period have been assumed to represent higher team speed, as compared to smaller changes in the same time period. But, this is not always the case (Boehm et al. 2000). However, in contrast to LOC measurements, our initial net size measurement considers changes to compiled instructions, or bytecode instructions, rather than lines of source code. Additionally, it measures the net size of a change, rather than the gross size of a change.

We chose to consider compiled instruction changes rather than LOC changes because the compiler removes organizational changes and allows us to focus on behavioral changes. That is, organizational changes such as alphabetizing methods within a class source file, commenting source lines, adding blank lines, and placing multiple statements on one line do not cause bytecode instruction changes when a class is compiled. And, although some more advanced LOC analysis tools might be capable of doing the same thing, we chose to use the compiler because it was simple for us to incorporate it into our study.

Additionally, we measure the net size of a developer's change, rather than the gross size of a developer's change, so that renaming of namespaces, classes, and methods are not counted twice—we also consider these to be organizational changes rather than behavioral changes. We want our measure to recognize that the simple renaming of a class resulted in X compiled instructions being added to the product and X compiled instructions being deleted from the product, and that the net size of the change was 0 (rather than $2X$).

To determine the initial net size of a developer's change set, we also used the jeanda tool. First, it models the classes and methods from before the change. Second, it models the classes and methods from the change. Third, it compares each of the methods to determine which compiled instructions have been added, changed, and deleted. Then, it calculates the net size of the change set, in compiled instructions, by summing the differences for the individual methods and classes.

Nonetheless, we realize that the initial net size of a developer's change only represents one attribute of a team's speed; it is missing the time dimension. That is why we also estimated discards over time—we also wanted to measure the changes that persisted because of a developer's change (and presumably continued to add value).

Dependent Variable—Effect on Speed (Discards over Time) As illustrated in our Java example (Section 3), and defined in our software change model, each product revision introduces two subsets of changes to the product: a subset of changes that will persist within the product at least until a certain time, and a subset that will be discarded before that. Therefore, by measuring the fraction of a change set that is discarded over time, we can adjust the initial net size measurement and determine a net size over time.

For example, consider a product revision which adds two product methods. As well, consider another product revision which adds four product methods. Assume that each of the compiled methods has ten bytecode instructions. If after six months, two product methods have persisted from the first revision and only one product method has persisted from the second, then 0% of the first revision has been discarded over time and 75% of the second revision has been discarded over time. Further, the first revision has a bigger net size over time (20 bytecode instructions added per 6 months) than the second revision (10 bytecode instructions added per 6 months), even though the second revision has a bigger initial net size (40 bytecode instructions as compared to 20 bytecode instructions).

To determine the fraction of a developer's change set that was discarded, we also used the jeanda tool. First, it compares the changes between the current revision and a future revision which is at least 150 days beyond the next revision—we also performed an auxiliary study (Bannerman and Martin 2010) which indicated that 80–90% of discards over time occur within 150 days of the initial change and that few discards occur after that.¹ Second, it compares the changes between the next revision and that same future revision. If more changes were persisted by the current revision as compared to the next revision, both relative to the future revision, then the current revision is credited with providing them. Any difference between what the current revision provided initially and what it provided persistently is the fraction that was discarded.

Dependent Variable—Effect on Quality (Potential Bugs) Product revisions may introduce or remove actual bugs in the classes and methods that change. However, direct automatic measurement of actual bugs is difficult or impossible. But, if we measure the number of potential bugs before and after a product revision with a tool, we can estimate the effect of the product revision on the number of actual bugs.

For example, consider a potential bug type named “double assignment of field.” If the product has five potential bugs of this type before the change, and ten after the change, and the number of other potential bug types has remained the same, then the number of potential bugs has increased by five (+5.0).

To estimate the effect of a developer's change set on the number of bugs in a product revision, we used the findbugs (<http://findbugs.sourceforge.net>) tool. This tool executes a static analysis of compiled Java code to find potential bugs in three broad categories: correctness, bad practice, and dodgy (Ayewah et al. 2008). First, we isolated the changed product classes from before the change and executed findbugs against them. Second, we isolated the changed product classes from the change and then similarly executed findbugs against them. Thus, the number of potential bugs increased if the second measurement was bigger than the first; for this study, we did not distinguish between the categories of bugs.

Dependent Variable—Effect on Quality (Average Method Complexity) Similar to the change in the number of potential bugs, product revisions may increase or decrease the average method complexity of the classes that change. Therefore,

¹Our auxiliary study also analyzed data from six projects, two of which were also studied in our main study.

if we measure the average method complexity before and after a product revision, we can determine the effect of the product revision on the average method complexity.

For example, consider the cyclomatic complexity of a method. If the complexity is ten before the change, and five after the change, and none of the other methods has changed, then the method complexity has decreased by five (-5.0). And, if the class contains two other methods, each with a method complexity of 10.0, then the average method complexity for the class has decreased from 10.0 ($30.0 / 3$) to 8.33 ($25.0 / 3$), which is a change of -1.67 .

To determine the effect of a developer's product revision on the complexity of the product, we used the cyvis (<http://cyvis.sourceforge.net>) tool. This tool executes a static analysis to measure the cyclomatic complexity (McCabe 1976) of methods of compiled Java code. First, we isolated the changed classes from before the change and then measured the complexities of their methods to calculate the old average method complexity. Second, we isolated the changed classes from the change and then measured their method complexities to calculate the new average method complexity. Thus, the complexity increased if the new average method complexity was bigger than the old average method complexity.

4.3.2 Analysis Procedures

Before analyzing our data, we had to relate the data from our independent variable to each of our dependent variables. We related the data by writing custom scripts to select and join the pertinent information from each of the individual result sets. For example, to determine a discard over time measurement for revision 200 of a particular product, we selected the particular row from our degree of TWD data for revision 200, selected the particular row from our discard over time data for revision 200, and joined the pertinent columns from each row.

Once we had the related data, we utilized the R project for Statistical Computing (<http://www.r-project.org>) to support our analysis of it, in accordance with the preliminary guidelines for empirical research (Kitchenham et al. 2002). First, we used it to provide a descriptive analysis of our data; we used it to create scatterplots, histograms, and boxplots of the data and to identify potential extreme outliers. Second, we used it to provide an inferential analysis of our data; we used it to compare and analyze the differences between the means for the three categories of TWD and their related effects.

Based on the descriptive analyses of our data, we identified the extreme outliers after transforming the data (if necessary) and then applying a thresholding rule. Generally, the data was not skewed but it did exhibit excess kurtosis (usually centered around zero); therefore, when appropriate we transformed the data into more normal distributions with log transformations. Then, we applied a thresholding rule—we considered the outliers to be extreme if they were outside three standard deviations from the mean.

After identifying the extreme outliers, we investigated their causes to convince ourselves that they could be considered outside of “normal,” so that we could justify excluding them from our data. We identified several causes for these extreme outliers:

- Major reorganizations of the product (for example, namespace changes).

- Merging two streams of product code (for example, incorporating “sandbox” code into the trunk stream once it was considered ready).
- Moving from one version of the JDK to another (for example, from 1.3 to 1.4)—this occurred at most two times per project we studied.
- Multiple developments that were “queued up” because of compilation problems (for example, several or many commits were made while the build was broken).
- Removing part of the product (for example, part of the product had been extracted and was now available as a third party library).

Since none of these infrequent activities has any bearing upon the degree of TWD that a developer practices, and since all of the extreme outliers were due to one of these causes, we excluded the extreme outliers.

To compare and analyze the differences between the means for the effects of the three categories of TWD (none, some, all), we used the Welch Two Sample unpaired t-test because the scatterplots and boxplots indicated that the variances were different and the data was not paired. Before doing so, we transformed the discards over time data sets from non-normal distributions to normal distributions, via bootstrapping, so that we could determine the confidence intervals for the differences between their means, in addition to their p-values; the other data sets were already in normal form. For the Welch Two Sample unpaired t-test, we used the 90% confidence level.

4.4 Summary

We were motivated to execute an empirical study to explore and describe the effects of TWD product changes on team speed and product quality. Based on our motivation, we decided to execute a multiple comparative study. Further, we decided to study open-source projects by mining their repositories, because they contain a wealth of freely available information.

Once we had decided to execute our study, we developed our plan for it incrementally as we iterated over what we wanted to achieve, the frame of reference for our study, what we wanted to study, and how we wanted to collect our data (Robson 2002; Runeson and Host 2009). We settled on exploring and describing the degree of TWD practiced by a team and the effects of the related changes on specific attributes related to team speed (initial net size and discards over time) and product quality attributes (potential bugs and average method complexity). Then, we evolved our study plan and our study protocol to facilitate collection and analysis of valid data.

5 Study Results

As described in Section 4, we rebuilt the revisions of the software products for our subject projects, and then measured the degree of TWD and the attributes related to team speed and the attributes related to product quality, for each set of changes that developers had made to those products. Then, we correlated the measurements and analyzed the differences between the means, as well as the statistical significances of the differences. In this section, we summarize and interpret the analysis results. In the next section, we examine the threats to the validity of the results.

To keep this results section focused (Robson 2002), we have not included the scatterplot, histogram, and boxplot diagrams of the measurements. For the same reason, we have not included tabular summaries for each of the projects; they are available in our technical report (Bannerman and Martin 2010). Instead, we present lattice plots to summarize the mean values for each of the measures and the confidence intervals for each of the comparisons (except for the net size over time measure, which is derived from the initial net size and the discards over time measures). Finally, we present the percent differences for some of the comparisons, to highlight significant differences.

5.1 Projects Studied

The projects we studied were: (1) ant; (2) cayenne; (3) commons-codec; (4) ehcache; (5) hadoop-core; and (6) xstream. Each of these are open-source projects that have been developed over many years. More details about them, and the data that we gathered from them, are available in our technical report.

5.2 Initial Net Size

Figure 1 summarizes the average initial net size of the changes that developers made, in terms of method instructions, for each project. These results show that when these developers exercised some of their changes with tests, they made bigger changes than when they exercised none of their changes with tests. Similarly, these results

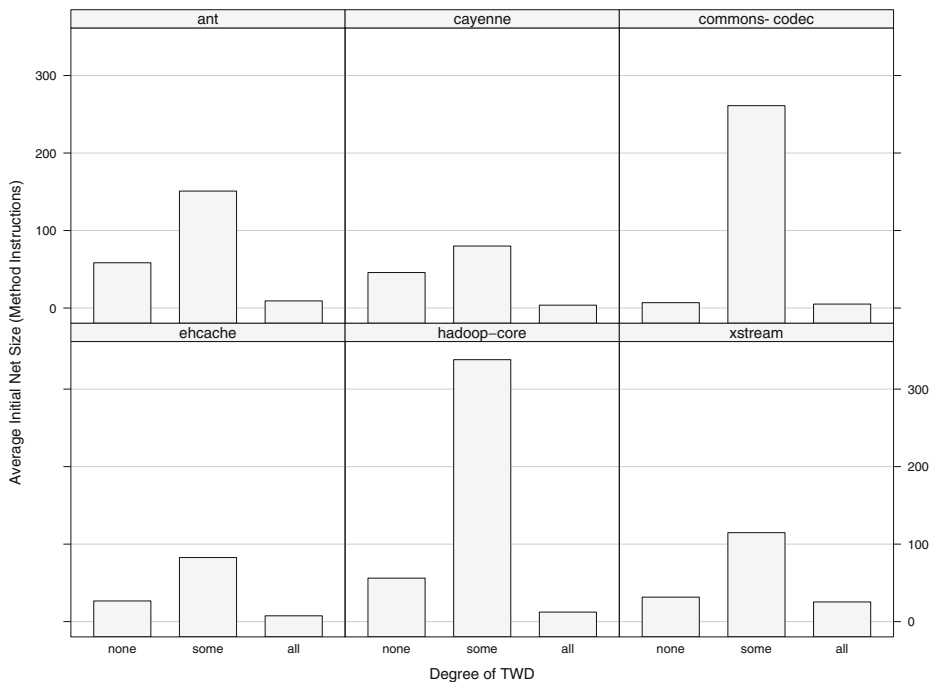


Fig. 1 Average initial net size

show that when these developers exercised all of their changes with tests, they made similarly sized or smaller changes than when they exercised none of their changes with tests.

Figure 2 summarizes the confidence intervals for the differences between the some-none and the all-none initial net size measurement comparisons. It also indicates the significances of the differences, with shading, according to the vertical color key on the far right of the figure. The significant confidence intervals confirm that the developer changes had bigger initial net sizes when they exercised some of their changes with tests, as compared to when they exercised none of their changes with tests; typically, the intervals indicate that those changes were between 0 and 100 method instructions bigger. They also confirm that the developer changes had smaller initial net sizes when they exercised all of their changes with tests, as compared to when they exercised none of their changes with tests; typically, the intervals indicate that those changes were between 0 and 100 method instructions smaller.

Figure 3 summarizes the percentage differences between the averages for the some-none and the all-none initial net size measurement comparisons, relative to the none measurements. These results show that when these developers exercised some of their changes with tests, on average they made 75–500+% bigger changes than when they exercised none of their changes with tests. Similarly, these results show that when these developers exercised all of their changes with tests, on average

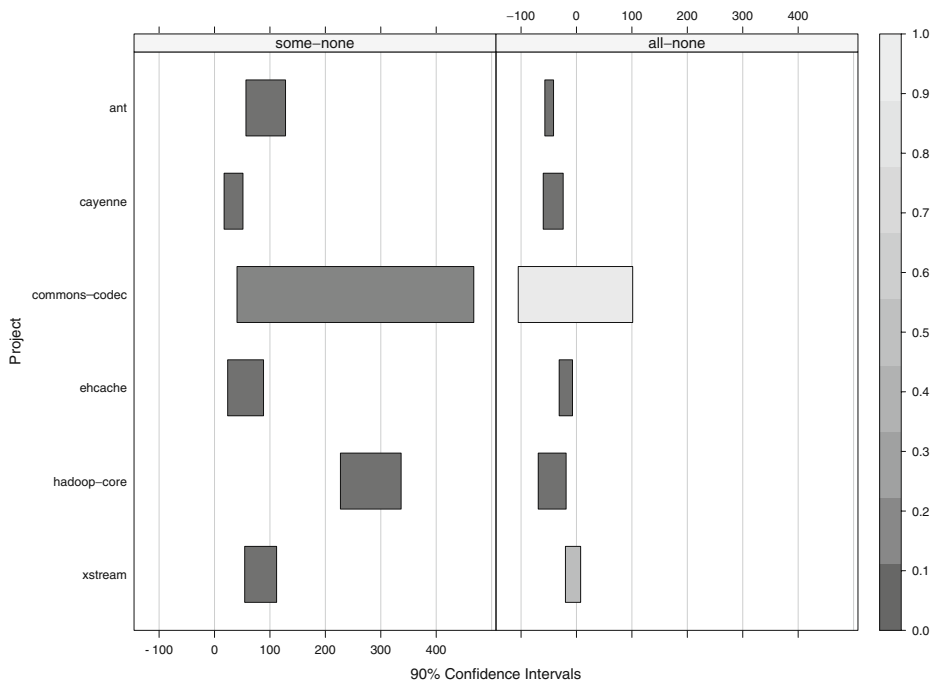


Fig. 2 Confidence intervals for initial net size comparisons

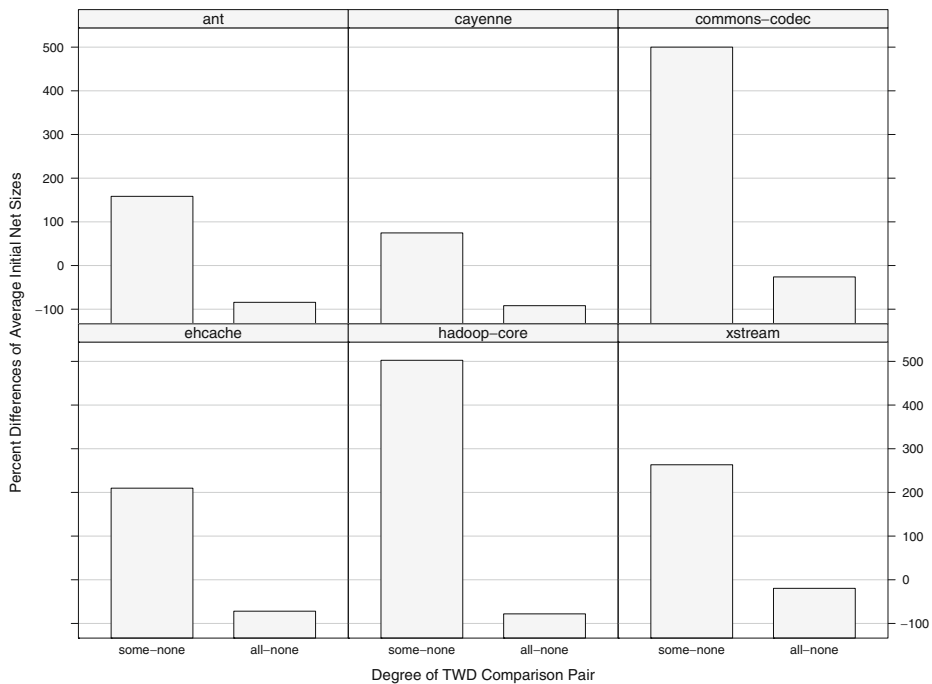


Fig. 3 Percentage differences for average initial net size comparisons

they made 25–100% smaller changes than when they exercised none of their changes with tests.

5.3 Discards Over Time

Figure 4 summarizes the discards over time for the changes that developers made, in terms of the percentages of the initial changes that were discarded, for each project. These results show that between 0 and 45% of the initial changes were discarded over time. However, there is no pattern that corresponds to the degree of TWD that the developers practiced. When these developers exercised some of their changes with tests, sometimes more and sometimes less of their changes were discarded over time, as compared to when they exercised none of their changes with tests. The same is true for when they exercised all of their changes with tests, as compared to when they exercised none of their changes with tests.

Figure 5 summarizes the confidence intervals for the differences between the some-none and the all-none discards over time measurement comparisons. It also indicates the significances of the differences, with shading, according to the vertical color key on the far right of the figure. All but one of the confidence intervals are significant, indicating that there were significant differences between the groups that were compared. However, in both the some-none and all-none comparison groups, there are cases where developers exercised their changes with tests and fewer of their changes were discarded over time (cayenne in the all-none comparison

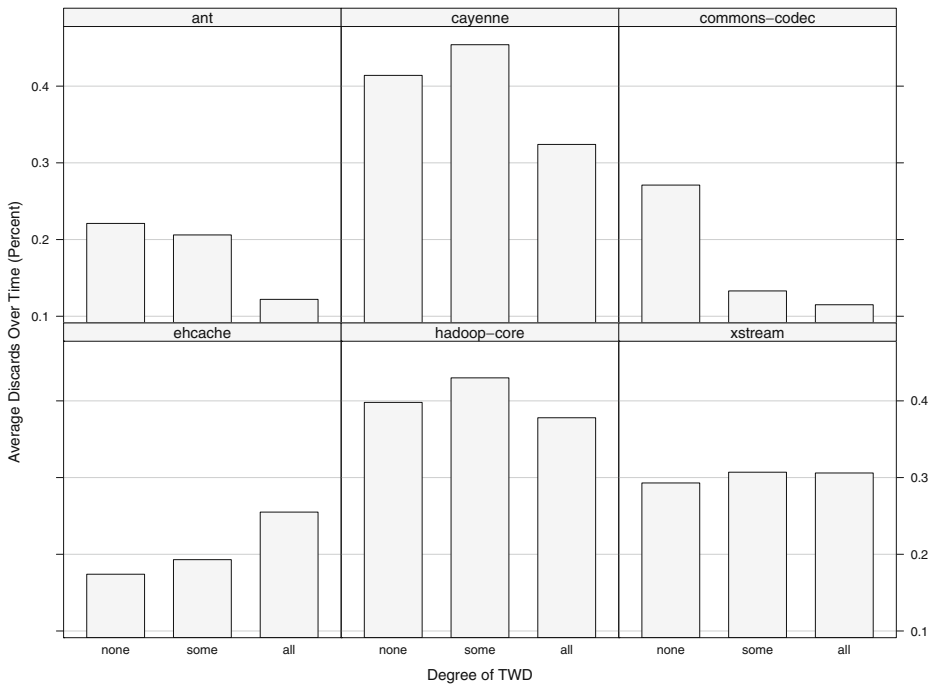


Fig. 4 Average discards over time

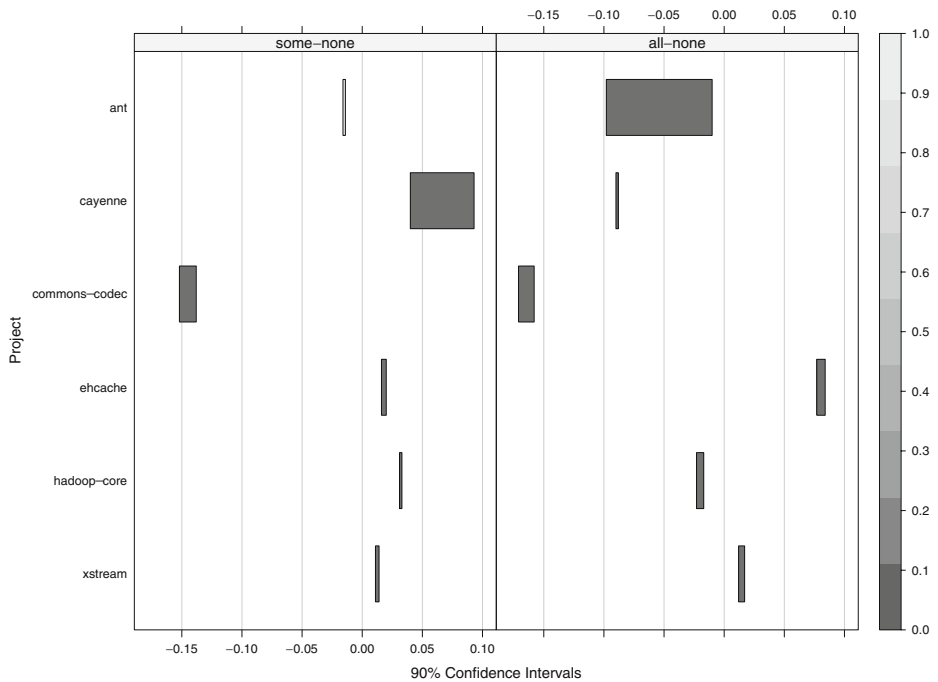


Fig. 5 Confidence intervals for discards over time comparisons

group) and cases where developers exercised their changes with tests and more of their changes were discarded over time (hadoop-core in the some-none comparison group). Additionally, these results show that the differences between the discards over time typically varied between $\pm 10\%$, for the projects that we studied.

5.4 Net Size Over Time

Figure 6 summarizes the net size over time of the changes that developers made, derived from the average initial net size and the average discards over time values, for each project. These results show that when these developers exercised some of their changes with tests, they made bigger changes than when they exercised none of their changes with tests. Similarly, these results show that when these developers exercised all of their changes with tests, they made smaller changes than when they exercised none of their changes with tests. So, even though there was up to a 10% difference with respect to the percentage of changes that were discarded over time, the pattern from the results for the initial net size comparisons remained similar over time.

Figure 7 summarizes the percentage differences between the derived values for the some-none and the all-none net size over time measurement comparisons, relative to the none measurements. These results show that when these developers exercised some of their changes with tests, on average they made 50–500+% bigger changes than when they exercised none of their changes with tests. Similarly, these results show that when these developers exercised all of their changes with tests, on average

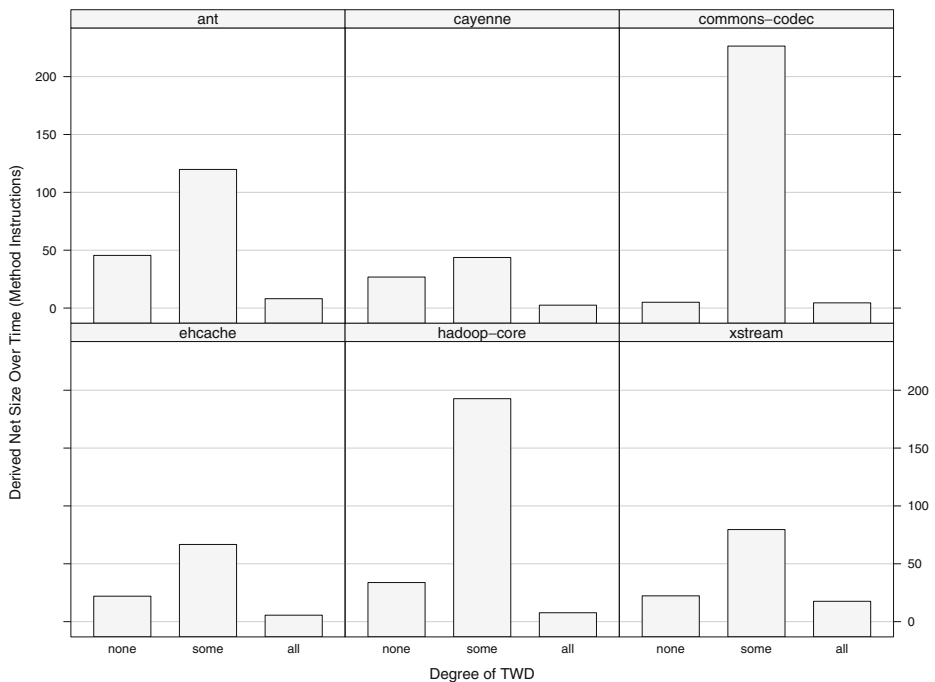


Fig. 6 Derived net size over time

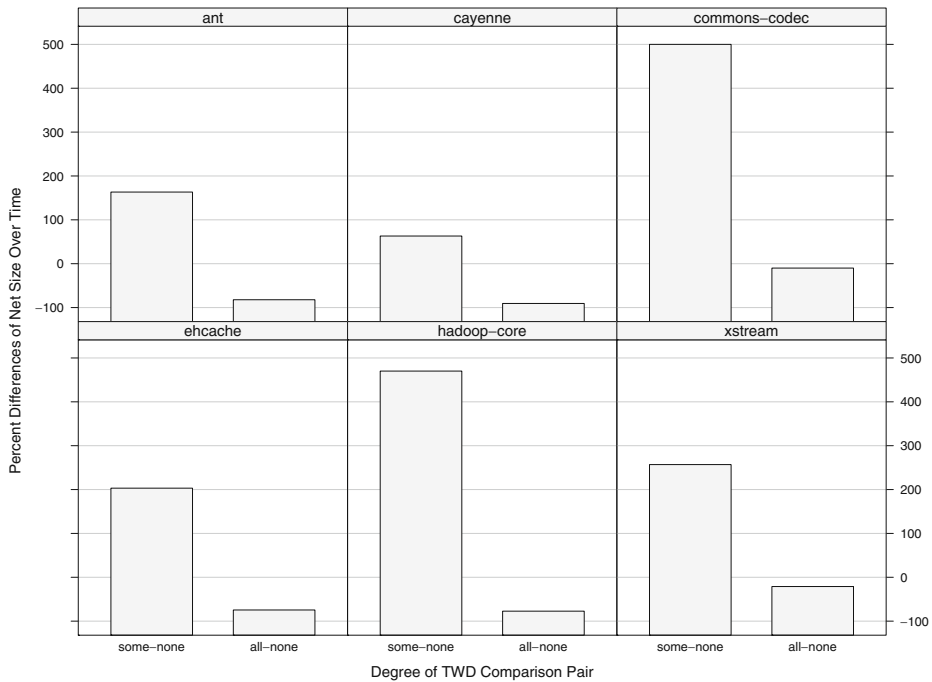


Fig. 7 Percentage differences for net size over time comparisons

they made 20–100% smaller changes than when they exercised none of their changes with tests.

5.5 Potential Bugs

Figure 8 summarizes the change in the potential bugs for the changes that developers made, for each project. These results show that, on average, all of the types of changes resulted in fewer potential bugs; whether the developers practiced TWD or not, there were fewer potential bugs in the affected code after they made their changes. However, these results do not identify a clear pattern where the developers practiced TWD, to some degree or fully, and removed more or less potential bugs than when they did not practice it.

Figure 9 summarizes the confidence intervals for the differences between the some-none and the all-none change in potential bugs measurement comparisons. It also indicates the significances of the differences, with shading, according to the vertical color key on the far right of the figure. Only two of the confidence intervals are significant. And, most of the confidence intervals straddle 0. Therefore, the comparisons do not indicate more or less potential bugs after a developer has made a change, practicing TWD to some degree or fully, both as compared to not practicing TWD at all. But, if we interpret the trend in these intervals, they indicate that practicing TWD to some degree or fully when making a change, as compared to not practicing it, may result in 0–1 fewer potential bugs being removed by the change.

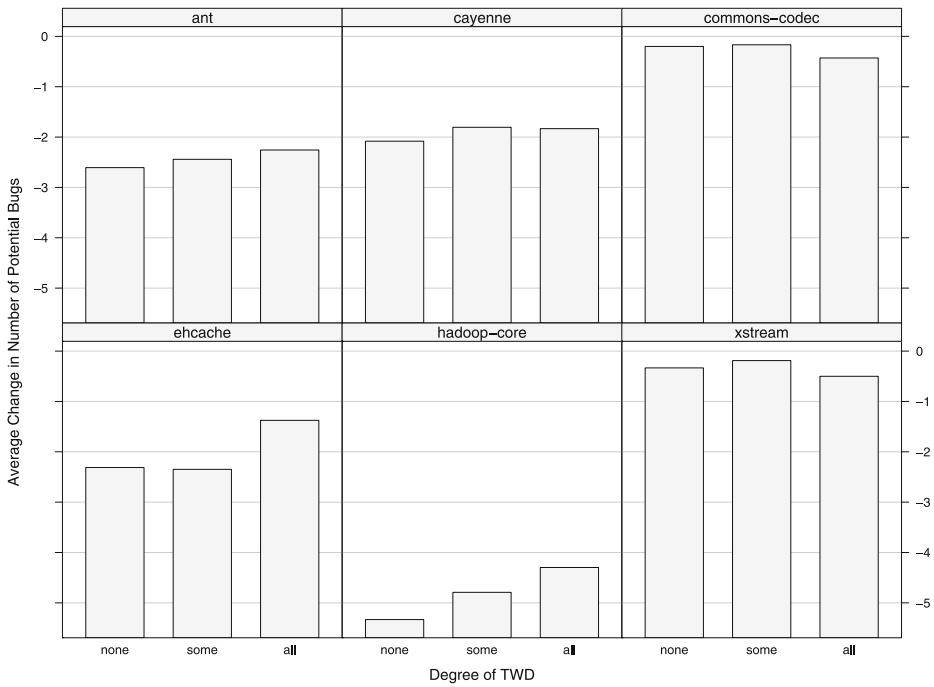


Fig. 8 Average change in potential bugs

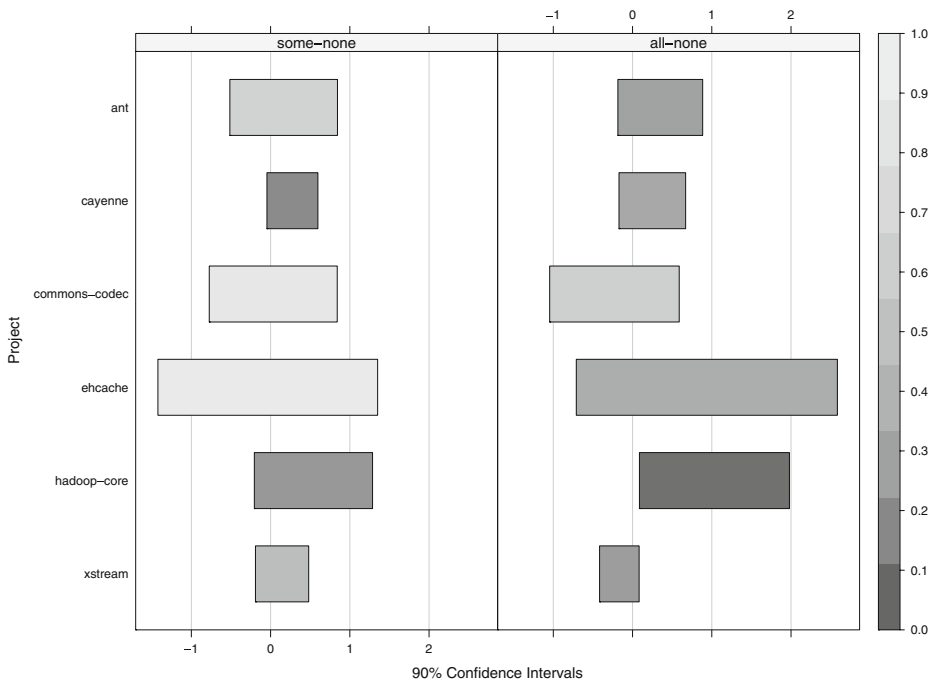


Fig. 9 Confidence intervals for change in potential bugs comparisons

5.6 Average Method Complexity

Figure 10 summarizes the average change in the average method complexity for the changes that developers made, for each project. These results show that when these developers exercised some of their changes with tests, on average they reduced the average method complexity of the methods that they changed. In contrast, the results also show that these developers, on average, increased the average method complexity of those methods when they exercised none of their changes with tests. Also in contrast, the results show that these developers, on average, sometimes increased and sometimes decreased the average method complexity of the methods that they changed when they exercised all of their changes with tests.

Figure 11 summarizes the confidence intervals for the differences between the some-none and the all-none change in average method complexity measurement comparisons. It also indicates the significances of the differences, with shading, according to the vertical color key on the far right of the figure. Three significant confidence intervals and three other consistent confidence intervals in the some-none comparison indicate that the developer changes reduced the average method complexity when they exercised some of their changes with tests, as compared to when they exercised none of their changes with tests; typically, the intervals indicate that those changes were between 0 and 0.1 less complex. And, although there are no significant confidence intervals in the all-none comparison, these intervals seem to confirm the neutrality of developers making changes when they practice TWD fully,

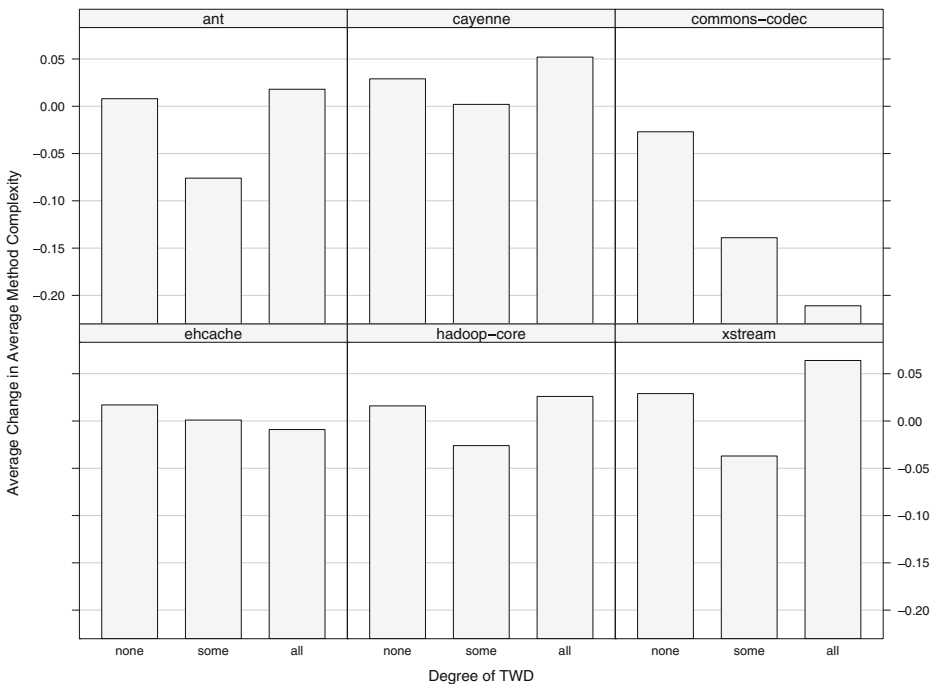


Fig. 10 Average change in average method complexity

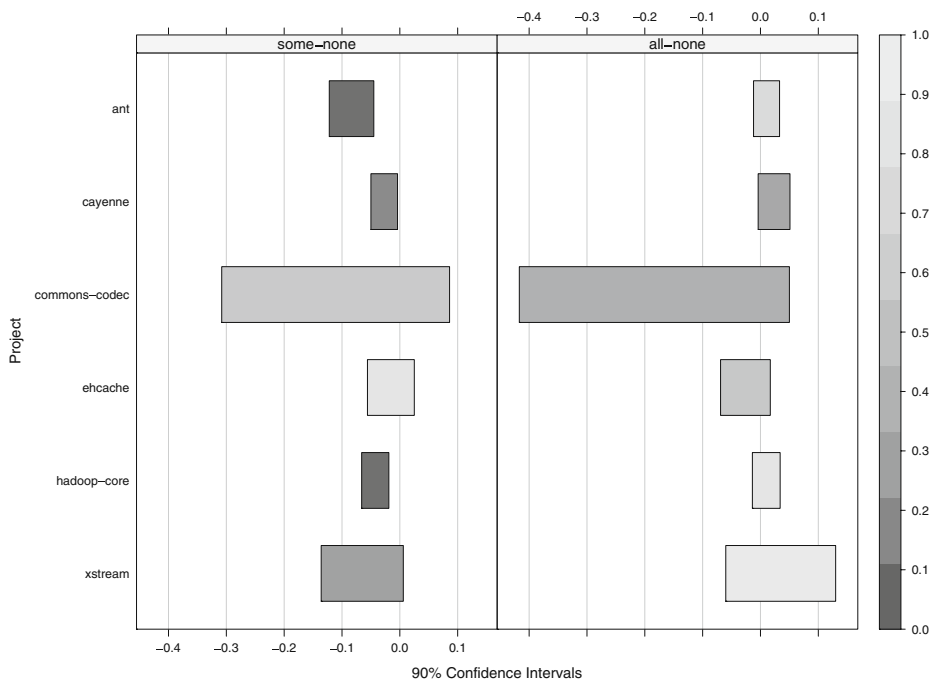


Fig. 11 Confidence intervals for change in average method complexity comparisons

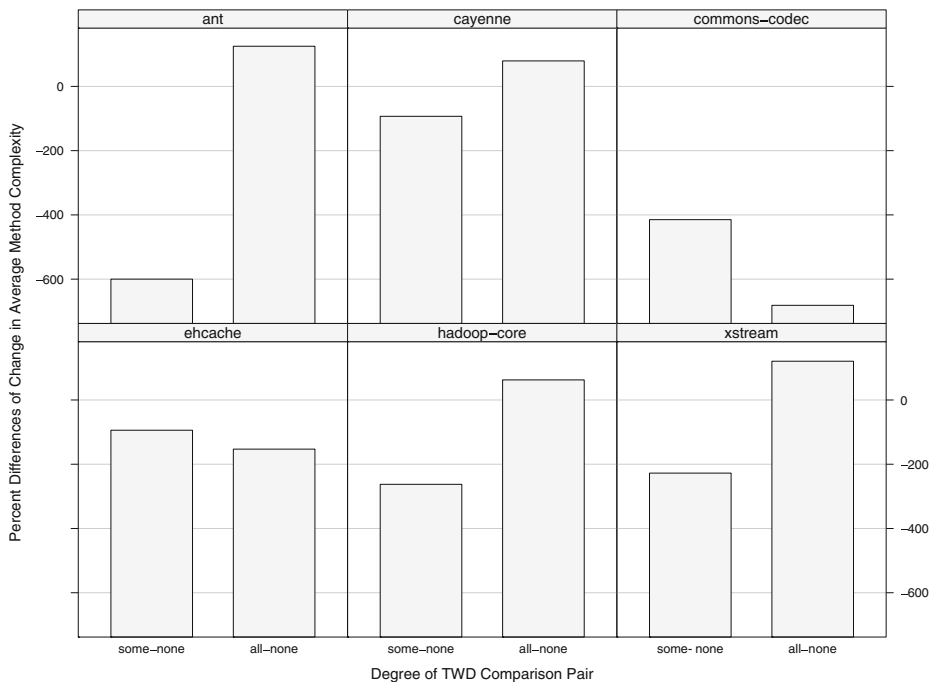


Fig. 12 Percentage differences for change in average method complexity comparisons

as compared to not practicing it at all; sometimes they increase the average method complexity of the methods they change and sometimes they decrease it.

Finally, Fig. 12 summarizes the percentage differences between the averages for the some-none and the all-none average method complexity measurement comparisons, relative to the none measurements. These results show that when these developers exercised some of their changes with tests, on average they reduced the average method complexity of the methods they changed by 50–600%, as compared to when they exercised none of their changes with tests. However, these results also show that when these developers exercised all of their changes with tests, on average they reduced or increased the average method complexity of the methods they changed by 200–600% or 50–150% respectively, as compared to when they exercised none of their changes with tests.

6 Threats to Validity

6.1 Compilation Instructions

Some reviewers of our study identified a threat to the validity of its data. Specifically, they suggested that unstable compilation instructions could have introduced noise into the data. Below, we describe the threat, our analysis to remove it, and the results of our analysis.

If the compilation instructions for a project are not stable, then noise may be introduced into the data. When compilation instructions and developer changes intersect, compiled instruction changes result from changes in the compilation instructions *and* from source code changes made by a developer. The compiled instruction changes resulting from the compilation instructions can be thought of as noise, whereas the changes resulting from the source code changes can be thought of as signal. Therefore, the signal to noise ratio in the data could degrade if the compilation instructions change too often or if the intensity is too great; the noise could obfuscate results or cause the results to be interpreted incorrectly. Specifically, increased noise due to changes in compilation, rather than actual developer changes, could have raised the initial net size and discards over time values. This may have made it more difficult to identify that practicing a certain degree of TWD results in bigger or smaller changes over time.

To remove this threat, we performed two types of analysis to confirm that the compilation instructions were stable for all of the project lifetimes that we studied. Initially, we gathered all of the revisions of the ant build files for the projects and compared the XML elements (and their compilation attributes) that the teams used to describe their Java compilation tasks. Following that, we replaced the Java class in the tools.jar that the ant javac task uses with a “decorator” which logged the actual compilation instructions, so that we could compare those compilation instructions from revision to revision as we rebuilt the revisions.

As a result of these comparisons, we found that the compilation instructions were very stable, with a couple of specific exceptions: changing versions of the JDK and one specific compilation task attribute for one revision in the ant project. Since we were studying projects over the course of many years, all of the projects made a change from one version of the JDK to another; plus, a couple of the projects

went through this transition twice. As well, when comparing the XML elements and their attributes, we also identified a single revision in the ant project (revision 267600) where the javac task's default deprecation attribute changed. Therefore, each project had at most three revisions where the compilation instruction noise was mixed together with the developer source code signal. As described in Section 4.3, when the data from these revisions was outside of the normal distribution, it was removed.

6.2 Static TWD Analysis

Another threat to the validity of our data is our measurement for the degree of TWD that a developer practiced while making a product revision (Section 4.3.1). Presently, our measure is determined by static analysis. This analysis gives the developer the “benefit of the doubt” by potentially overestimating the degree to which they practiced TWD—due to dynamic binding in Java we treat indirect references from an automated test method to a product method as if the product method would be exercised by the test method. It also treats a method as either completely exercised or not at all exercised, potentially crediting a developer with taking a TWD approach even if they have not exercised the new or changed paths within a particular changed product method.

These two problems could cause some data in our study to be mis-classified. A developer's product change that was made without exercising the product changes with automated tests could be interpreted as if the developer had practiced TWD to some degree, or even fully. Because previous automated tests were in place to exercise some or all of the changed methods, changes that should be classified as none could be classified as some or all. Similarly, changes that should be classified as some could be classified as all.

In practice, the first problem with static analysis is nearly moot; after analyzing all of the revisions of all of the projects we studied, 82% of the relationships from automated test methods to product methods are direct. Further, the other 18% of the relationships are from abstract automated test methods to abstract product methods; when we manually analyzed 50 of these, only two product change methods had been treated as if they had been exercised when they were not. These results are consistent with our industrial experience—most automated tests are written at the unit level and test a public method of that unit directly.

With respect to the second problem, we manually analyzed 100 samples of the changes that were made, across all projects, to see if we correctly classified them with respect to the degree of TWD that the developer actually practiced: none, some, or all. For 96 of these samples, our static analysis classification and our manual classification match. Nonetheless, as identified in the future work of Section 7, we think that the degree of TWD that a developer practiced should be determined with a dynamic approach.

Indeed, we would have preferred to have used a dynamic approach to measure the relationships between the automated product tests and the product. Rather than potentially overestimating the relationships and the path exercises, we could have measured the actual exercising of those changed product methods by executing the product tests. But, our early efforts to build and execute product tests against each product revision were stymied. We found doing so to be difficult or nearly impossible

for the projects we studied; often it was difficult to build the product so that it could be monitored and more often the tests would not execute (due to further environmental setup requirements that changed over time). Therefore, we accepted this limitation for this present study.

6.3 Tool Bias

A third threat to the validity of our data is tool bias, which could result from developers using the same tools as us (Section 4.3.1) to help them create bigger or smaller changes, reduce the number of potential bugs, or reduce the average method complexity. For example, if certain developers on a team were more prone to practice TWD and if they also used an additional tool to help them reduce the number of potential bugs when they made a change, this could confound the relationships between practicing TWD and its effects on the number of potential bugs.

To investigate this threat, we downloaded and searched the mail archives² for each of the projects that we studied. Our search results do not indicate that any of the projects used additional tools to help them create bigger or smaller changes (specifically jeanda). Similarly, our search results do not indicate that any of the projects used additional tools to help them reduce their average method complexity (specifically cyvis).

However, our search results do indicate that a couple of the developers from two of the projects, commons-codec and hadoop-core, discussed using the findbugs tool during the final year of our study of their projects. Since there were no significant results related to findbugs within the commons-codec project, there are no results to invalidate. But, within the hadoop-core project, there was a significant result which indicated that practicing TWD fully resulted in between 0 and 2 more potential bugs, on average. So, if we assume that developers using the findbugs tool would on average remove more potential bugs, as measured by findbugs, and we assume that the two hadoop-core developers used findbugs but did not practice TWD fully, then they may have created an artificial difference between themselves and the developers who did practice TWD fully. Therefore, this hadoop-core potential bugs significant result might be invalid.

6.4 Potential Bugs

Another threat to the validity of our data, specifically our potential bugs data, is the indirect relationship between practicing TWD and removing potential bugs, as defined by the findbugs tool. We and some of our study reviewers agree that developers that practice TWD are generally *not* intentionally and directly removing potential bugs from their product. And, we and they agree that it is more likely that the developers are practicing TWD to ensure that their product changes function properly and to give them confidence that they haven't broken anything. But, we think that developers that practice TWD might indirectly remove more potential bugs, because their increased confidence allows them to refactor and simplify their product, which could remove bugs that were introduced because of complexity.

²We used case-insensitive grep to search the archives for “cyvis,” “findbugs,” and “jeanda.”

In contrast, they are skeptical that practicing TWD could result in a significant reduction in potential bugs, even indirectly, because they feel that the types of potential bugs removed as a result of refactoring are different than the types of potential bugs found by the findbugs tool.

Our thinking was and is based on both our experience and some of the past study results. In our experience, practicing TWD can remove all sorts of potential problems, including bugs, from source code. And, several case studies have found a reduction in a variety of measures related to actual bugs when developers have practiced TFD: (1) defect density (Williams et al. 2003; Bhat and Nagappan 2006; Sanchez et al. 2007; Nagappan et al. 2008); (2) defect rate (Maximilien and Williams 2003); and (3) fault rate (Damm and Lundberg 2006). Therefore, we think it was reasonable to study a potential relationship between developers practicing TWD and the resulting changes in the number of potential bugs.

Their thinking is based on their experience and a lack of past study results specific to findbugs. In their experience, several other types of development activities are also likely to cause the removal of potential bugs (as defined by findbugs)—for example, design and review activities. And, they wonder whether developers would even agree with the findbugs definitions of potential bugs, which could result in a higher measure of potential bugs even when the developers themselves would think that they had removed potential bugs. Also, although past studies have found a reduction in measures related to actual bugs, they point out that none of them have found a reduction in potential bugs (again, as defined by findbugs). Therefore, they question whether it was reasonable to study this potential relationship.

So, even if our study had found significant relationships between TWD and potential bugs, these results would have been subject, due to the threats to validity that these reviewers suggest.

7 Conclusion

In this section, we present our conclusions, drawn from the study results in Section 5 and our interpretation of them. Additionally, we relate our results to the results of the previous studies on TFD, which we presented in Section 2. Then, we discuss the implications and limitations of our results. Finally, we suggest some potential future work that we have identified during the planning, execution, and reporting of this study.

7.1 Conclusions

Based on our study design (Section 4) and our study results (Section 5), when a developer practiced TWD while making product changes, the effects of those changes on the attributes related to team speed and product quality were significantly different than when they did not. Further, the degrees to which they practiced TWD corresponded with different effects on the specific attributes that we studied.

When they exercised some of their product changes with tests, on average they made significantly bigger product changes over time that resulted in significantly less average method complexity in the product. In contrast, when they exercised all of their product changes with tests, they made significantly smaller product

changes over time that did not significantly increase or decrease the average method complexity in the product. Finally, the number of potential bugs within the product was not significantly different according to the degree of TWD that a developer practiced.

These results agree in part and disagree in part with our initial hypotheses, in Section 4. The results for developers practicing TWD to some degree agree with our initial hypotheses: bigger changes over time and less average method complexity. However, the results for developers practicing TWD fully do not agree with our initial hypotheses: smaller changes over time with no significant differences in average method complexity. Finally, we did expect that practicing TWD to some degree, and fully, would decrease the number of potential bugs; our results do not support that conclusion.

We interpret these results to indicate that automated testing of product changes, *before* sharing those changes with the rest of the team, enabled the teams that we studied to reduce their product's internal complexity more quickly than when they did not. Further, we interpret these results to indicate that the teams that we studied reduced their product's internal complexity incrementally, rather than all at once.

We interpret the bigger changes over time and the lower average method complexity to be the result of refactoring; when a developer has automated tests to support refactoring, they are more likely to simplify their product and remove duplication from it because they have more confidence that they will not break anything. And, we interpret the smaller changes over time to be the ultimate benefit of reducing the product's complexity; once areas of a product have been simplified then only smaller changes will be necessary to achieve the desired effects.

Therefore, based on these interpretations, we conclude that teams can benefit from practicing TWD. In particular, we conclude that they can reduce their product's complexity more quickly when they practice TWD, incrementally adding automated tests to support its simplification.

7.2 Relation to Existing Studies

The existing studies on TFD are either case studies or controlled experiments which have compared the effects of TFD between projects and products, rather than within a product. That is, the unit of analysis has been the product, and how TFD affected it as compared to another, rather than how it affected the changes to the same product over time. Additionally, each of the studies, including our own, has studied different attributes related to team speed and product quality (see Section 2, Table 3). Finally, we have studied TWD, which includes TFD and two additional variants that also describe a developer delivering fine-grained changes to their product in addition to tests for those changes. These differences make it difficult to compare our results to the results of the other studies.

Nonetheless, when we do compare them, our results match some parts of the other studies' results. Our results for developers practicing TWD to some degree match the increase in attributes related to speed from the results of the student studies (Section 2, Table 1) and the increase in attributes related to quality from the results of the professional studies (Section 2, Table 2). In contrast our results for developers practicing TWD fully match the decrease in attributes related to speed from the results of the professional studies.

7.3 Implications and Limitations

One implication of our study is that the extra effort (perceived or actual) involved in maintaining automated tests can be justified. If bigger changes over time and reducing the average method complexity are deemed to be worth the extra effort, then a team should practice TWD to some degree. This implication might be particularly useful to a team that knows they need to make many changes over time to simplify a complex system. Plus, if eventual stability is valued by a team, practicing TWD may allow them to get to the point where all product changes are exercised by automated tests and thus facilitate smaller changes over time.

Another implication of the study is that a team might benefit from testing their fine-grained product changes, even if they write their automated product tests along with or after their product changes. That is, they might benefit from TWD even if they do not practice TFD in a strict manner—it may be more effective to develop tests *with* product changes than to develop them *before* product changes.

However, our study design prevents us from proving (statistically) that practicing TWD to a degree is the cause for the significant effects that we observed. Rather, we have only been able to prove (statistically) that there are significant differences between the means (averages) for some of the effects we studied—in particular, the net size over time and the average method complexity. This is the main limitation of our study.

As well, we would not yet recommend generalizing beyond the Java open-source population. Although we have no particular reasons to believe that the results may differ significantly, differences such as technology, experience, product type, and timelines could yield conflicting results. Hopefully, future studies can expand the population for which these results are valid and identify the boundaries of those where they are not.

7.4 Future Work

The most compelling question raised by this study is whether practicing TWD to a degree is the cause (or a cause) of the significantly different effects we observed on the attributes related to team speed and product quality. Therefore, we submit that this is an important area of future work.

As well, as described in Section 5, we think that measuring the degree of TWD more accurately is an important area of future work. That is, rather than measure the degree of TWD with a static analysis tool, we think that future work should attempt to measure it with a dynamic analysis tool (such as Clover, Cobertura, or Emma).

In addition, this study has identified that a team can benefit from developing and delivering product tests *with* their product changes. That is, they can benefit from practicing TWD, which may or may not require practicing TFD. Therefore, we submit that an important area of future work will be determining whether the benefits of TWD come from developing product tests first (as in TFD), or just from developing product tests with product changes, as opposed to TLD.

We submit that another important area of future work is identifying the boundaries of the external validity for our results. For example, do these results hold if we expand to tens or hundreds of Java open-source projects? And, are the results similar for C# .NET open-source projects and for “closed-source” projects?

A final important area of future work is identifying the scope of the potential benefits of TWD. Are the potential benefits limited to increased net size over time and decreased average method complexity or are there others? And, are there potential detriments? To find out, we or others will need to study the effects of practicing TWD on other attributes of team speed or product quality. Additionally, is there a profile with which a team could optimize the potential benefits of TWD by practicing it to a certain degree, such as 25%, 50%, 75%, or 100%? Further, should a team practice TWD to different degrees, according to the current complexity of the product, or should they always practice it to a certain degree?

7.5 Summary

We began our study with this general research question: *What are the relationships between practicing TWD, to varying degrees, and two attributes related to team speed (initial net size and discards over time) and two attributes related to product quality (potential bugs and average method complexity), when those practicing it are real-life software development teams, developing their product over the course of many years?*

In preparation for answering this question, we developed a precise software change model (Bannerman and Martin 2010) so that we could determine the degree of TWD, the initial net size, and the discards over time for a particular developer's change. Then, we calibrated that model by performing an auxiliary study (Bannerman and Martin 2010) to determine how far into the future we needed to “look” to estimate the effect on one attribute related to a team's speed: discards over time—150 days was the answer. After that, we planned and designed our main study to explore and describe the relationships between practicing TWD when making a change and the effects of the change on the two attributes related to team speed and two attributes related to product quality (Section 4). Finally, we executed our main study, assembled its results, and analyzed its validity (Section 5).

Our main study results indicate that there were significant differences between developers practicing TWD when making a change, and the resulting effects on the attributes that we studied. When the developers exercised some of their changes with tests, they made significantly larger changes over time while significantly reducing their product's average method complexity. And, when they exercised all of their changes with tests, they made significantly smaller changes over time but did not significantly affect their product's average method complexity. However, in neither case did they significantly affect the number of potential bugs within their changes.

We interpret these results to indicate that developers refactored their product more when they developed their tests with their product changes, reducing the complexity of their product faster. Therefore, we conclude that teams that wish to reduce the complexity of their product over time, and to do so more quickly, can benefit from practicing TWD.

References

- Abrahamsson P, Hanhineva A, Jaalinoja J (2005) Improving business agility through technical solutions: a case study on test-driven development in mobile software development. In: IFIP 2005: business agility and information technology diffusion, pp 1–17
- Auer K, Miller R (2002) Extreme programming applied. Addison-Wesley, Boston

- Ayewah N, Hovemeyer D, Morgenthaler JD, Penix J, Pugh W (2008) Using static analysis to find bugs. *IEEE Softw* 25(5):22–29. doi:[10.1109/MS.2008.130](https://doi.org/10.1109/MS.2008.130)
- Bannerman S, Martin A (2010) A multiple comparative study of test-with development product changes and their effects on team speed and product quality. Tech. Rep. RR-10-03, The University of Oxford. <http://www.comlab.ox.ac.uk/techreports/cs/2010.html>
- Basili VR, Selby RW, Hutchens DH (1986) Experimentation in software engineering. *IEEE Trans Softw Eng* 12(7):733–743
- Basili VR, Shull F, Lanubile F (1999) Building knowledge through families of experiments. *IEEE Trans Softw Eng* 25(4):456–473. doi:[10.1109/32.799939](https://doi.org/10.1109/32.799939)
- Beck K (1999a) Embracing change with extreme programming. *IEEE Comput* 32(10):70–77
- Beck K (1999b) Extreme programming explained: embrace change. Addison-Wesley
- Beck K (2003) Test-driven development. Addison-Wesley, Boston
- Benbasat I, Goldstein DK, Mead M (1987) The case research strategy in studies of information systems. *JSTOR MIS Q* 11(3):369–386. doi:[10.2307/248684](https://doi.org/10.2307/248684)
- Bhat T, Nagappan N (2006) Evaluating the efficacy of test-driven development: industrial case studies. In: *ISESE '06: proceedings of the 2006 ACM/IEEE international symposium on empirical software engineering*. ACM, New York, pp 356–363. doi:[10.1145/1159733.1159787](https://doi.org/10.1145/1159733.1159787)
- Boehm BW, Abts C, Brown AW, Chulani S, Clark BK, Horowitz E, Madachy R, Reifer DJ, Steece B (2000) Software cost estimation with Cocomo II. Prentice Hall, Englewood Cliffs
- Canfora G, Cimitile A, Garcia F, Piattini M, Visaggio CA (2006) Evaluating advantages of test driven development: a controlled experiment with professionals. In: *ISESE '06: proceedings of the 2006 ACM/IEEE international symposium on empirical software engineering*. ACM, New York, pp 364–371. doi:[10.1145/1159733.1159788](https://doi.org/10.1145/1159733.1159788)
- Card DN, McGarry FE, Page GT (1987) Evaluating software engineering technologies. *IEEE Trans Softw Eng* 13(7):845–851. doi:[10.1109/TSE.1987.233495](https://doi.org/10.1109/TSE.1987.233495)
- Cook JE, Votta LG, Wolf AL (1998) Cost-effective analysis of in-place software processes. *IEEE Trans Softw Eng* 24(8):650–663. doi:[10.1109/32.707700](https://doi.org/10.1109/32.707700)
- Damm LO, Lundberg L (2006) Results from introducing component-level test automation and test-driven development. *J Syst Softw* 79(7):1001–1014. doi:[10.1016/j.jss.2005.10.015](https://doi.org/10.1016/j.jss.2005.10.015)
- Diehl S, Hassan AE, Holt RC (2005) Report on msr 2005: international workshop on mining software repositories. *SIGSOFT Softw Eng Notes* 30(5):1–3. doi:[10.1145/1095430.1095433](https://doi.org/10.1145/1095430.1095433)
- Diehl S, Gall H, Pinzger M, Hassan AE (2006) Msr 2006: the 3rd international workshop on mining software repositories. In: *ICSE '06: proceedings of the 28th international conference on software engineering*. ACM, New York, pp 1021–1021. doi:[10.1145/1134285.1134483](https://doi.org/10.1145/1134285.1134483)
- Erdogmus H, Morisio M, Torchiano M (2005) On the effectiveness of the test-first approach to programming. *IEEE Trans Softw Eng* 31(3):226–237. doi:[10.1109/TSE.2005.37](https://doi.org/10.1109/TSE.2005.37)
- Fenton N, Pfleeger S (1997) Software metrics: a rigorous and practical approach. PWS Publishing Company, Boston
- Gall H, Lanza M, Zimmermann T (2007) 4th international workshop on mining software repositories (msr 2007). In: *ICSE COMPANION '07: companion to the proceedings of the 29th international conference on software engineering*. IEEE Computer Society, Washington, DC, pp 107–108. doi:[10.1109/ICSECOMPANION.2007.8](https://doi.org/10.1109/ICSECOMPANION.2007.8)
- George B, Williams L (2003) An initial investigation of test driven development in industry. In: *SAC '03: proceedings of the 2003 ACM symposium on applied computing*. ACM, New York, pp 1135–1139. doi:[10.1145/952532.952753](https://doi.org/10.1145/952532.952753)
- Geras A, Smith M, Miller J (2004) A prototype empirical evaluation of test driven development. In: *METRICS '04: proceedings of the software metrics, 10th international symposium*. IEEE Computer Society, Washington, DC, pp 405–416. doi:[10.1109/METRICS.2004.2](https://doi.org/10.1109/METRICS.2004.2)
- Gupta A, Jalote P (2007) An experimental evaluation of the effectiveness and efficiency of the test driven development. In: *ESEM '07: proceedings of the first international symposium on empirical software engineering and measurement*. IEEE Computer Society, Washington, DC, pp 285–294. doi:[10.1109/ESEM.2007.20](https://doi.org/10.1109/ESEM.2007.20)
- Hannay JE, Sjöberg DIK, Dyba T (2007) A systematic review of theory use in software engineering experiments. *IEEE Trans Softw Eng* 33(2):87–107. doi:[10.1109/TSE.2007.12](https://doi.org/10.1109/TSE.2007.12)
- Hassan AE, Holt RC, Mockus A (2005) Report on msr 2004: international workshop on mining software repositories. *SIGSOFT Softw Eng Notes* 30(1):4. doi:[10.1145/1039174.1039188](https://doi.org/10.1145/1039174.1039188)
- Janzen DS, Saiedian H (2006) On the influence of test-driven development on software design. In: *CSEET '06: proceedings of the 19th conference on software engineering education & training*. IEEE Computer Society, Washington, DC, pp 141–148. doi:[10.1109/CSEET.2006.25](https://doi.org/10.1109/CSEET.2006.25)

- Jeffries R, Anderson A, Hendrickson C (2001) *Extreme programming installed*. Addison-Wesley, Boston
- Kitchenham B, Pickard L, Pfleeger SL (1995) Case studies for method and tool evaluation. *IEEE Softw* 12(4):52–62. doi:[10.1109/52.391832](https://doi.org/10.1109/52.391832)
- Kitchenham B, Pfleeger S, Pickard L, Jones P, Hoaglin D, Emam KE, Rosenberg J (2002) Preliminary guidelines for empirical research in software engineering. *IEEE Trans Softw Eng* 28(8):721–734
- Lanza M, Godfrey MW, Kim S (2008) Msr 2008—5th working conference on mining software repositories. In: *ICSE companion '08: companion of the 30th international conference on software engineering*. ACM, New York, pp 1037–1038. doi:[10.1145/1370175.1370235](https://doi.org/10.1145/1370175.1370235)
- Lethbridge TC, Sim SE, Singer J (2005) Studying software engineers: data collection techniques for software field studies. *Empir Software Eng* 10(3):311–341. doi:[10.1007/s10664-005-1290-x](https://doi.org/10.1007/s10664-005-1290-x)
- Madeyski L (2005) Preliminary analysis of the effects of pair programming and test-driven development on the external code quality. In: *Proceeding of the 2005 conference on software engineering: evolution and emerging technologies*. IOS Press, Amsterdam, pp 113–123
- Maximilien EM, Williams L (2003) Assessing test-driven development at IBM. In: *ICSE '03: proceedings of the 25th international conference on software engineering*. IEEE Computer Society, Washington, DC, pp 564–569
- McCabe TJ (1976) A complexity measure. *IEEE Trans Softw Eng* 2(4):308–320
- Muller M, Hagner O (2002) Experiment about test-first programming. *IEE Proc, Softw* 149(5):131–136
- Nagappan N, Maximilien EM, Bhat T, Williams L (2008) Realizing quality improvement through test driven development: results and experiences of four industrial teams. *Empir Software Eng* 13(3):289–302. doi:[10.1007/s10664-008-9062-z](https://doi.org/10.1007/s10664-008-9062-z)
- Pancur M, Ciglaric M, Trampus M, Vidmar T (2003) Towards empirical evaluation of. test-driven development in a university environment. In: *EUROCON 2003: IEEE region 8 proceedings, vol 2*. IEEE Press, pp 83–86
- Pinsonneault A, Kraemer KL (1993) Survey research methodology in management information systems: an assessment. *J Manage Inf Syst* 10(2):75–105
- Raymond ES (1999) The magic cauldron. In: *The cathedral & the bazaar*. O'Reilly & Associates, pp 137–194
- Robson C (2002) *Real world research*. Blackwell Publishers, Cambridge
- Runeson P, Host M (2009) Guidelines for conducting and reporting case study research in software engineering. *Empir Software Eng* 14(2):131–164
- Sanchez JC, Williams L, Maximilien EM (2007) On the sustained use of a test-driven development practice at IBM. In: *AGILE '07: proceedings of the AGILE 2007*. IEEE Computer Society, Washington, DC, pp 5–14. doi:[10.1109/AGILE.2007.43](https://doi.org/10.1109/AGILE.2007.43)
- Sinialto M, Abrahamsson P (2007) A comparative case study on the impact of test-driven development on program design and test coverage. In: *ESEM '07: proceedings of the first international symposium on empirical software engineering and measurement*. IEEE Computer Society, Washington, DC, pp 275–284. doi:[10.1109/ESEM.2007.2](https://doi.org/10.1109/ESEM.2007.2)
- Sjoberg DIK, Hannay JE, Hansen O, Kampenes VB, Karahasanovic A, Liborg N, Rekdal AC (2005) A survey of controlled experiments in software engineering. *IEEE Trans Softw Eng* 31(9):733–753. doi:[10.1109/TSE.2005.97](https://doi.org/10.1109/TSE.2005.97)
- Williams L, Maximilien EM, Vouk M (2003) Test-driven development as a defect-reduction practice. In: *ISSRE '03: proceedings of the 14th international symposium on software reliability engineering*. IEEE Computer Society, Washington, DC, pp 34–45
- Zaidman A, Van Rompaey B, Demeyer S, van Deursen A (2008) Mining software repositories to study co-evolution of production & test code. In: *ICST '08: proceedings of the 2008 international conference on software testing, verification, and validation*. IEEE Computer Society, Washington, DC, pp 220–229. doi:[10.1109/ICST.2008.47](https://doi.org/10.1109/ICST.2008.47)



Steve Bannerman leads the Web Product Development Group for DMI's TimeClock Plus product. During the past twenty years, he has architected, designed, and developed in object-oriented development environments, including Smalltalk, Java, and C# projects in the telecommunications, financial, BPM, and time and attendance domains.

In addition to his industry work, he is interested in software engineering and its application to industrial projects. As a result of these interests, he is nearing completion of a doctorate in software engineering at the University of Oxford, where has been performing part-time empirical software engineering research. Prior to that, he completed a part-time masters in software engineering at Southern Methodist University.



Andrew Martin lectures to Software Professionals as part of Oxford University's Software Engineering Programme, where he directs the Master of Science course in Software and Systems Security. He has a background in formal methods, but today devotes most of his time to issues of security in distributed systems. He has been particularly interested in the grid computing paradigm, the security questions that raises, and how the technologies of trusted computing can help to address the challenges in that area.

Andrew wrote a doctoral thesis on the subject 'Machine-Assisted Theorem Proving for Software Engineering', in the early 1990s. He then worked as a Research Fellow in the Software Verification Research Centre at the University of Queensland, Australia. Returning to the UK, he was briefly a lecturer at the University of Southampton, before returning to Oxford to take up his present post in 1999. Dr Martin is a fellow of Kellogg College, Oxford.